

Laravel Framework

Unit 21

الصف الثالث



مدارس
للتكنولوجيا
التطبيقية

IT Department
Web Programming

Laravel Framework
Workshops

1^{st.}

2025 - 2026

Laravel Framework

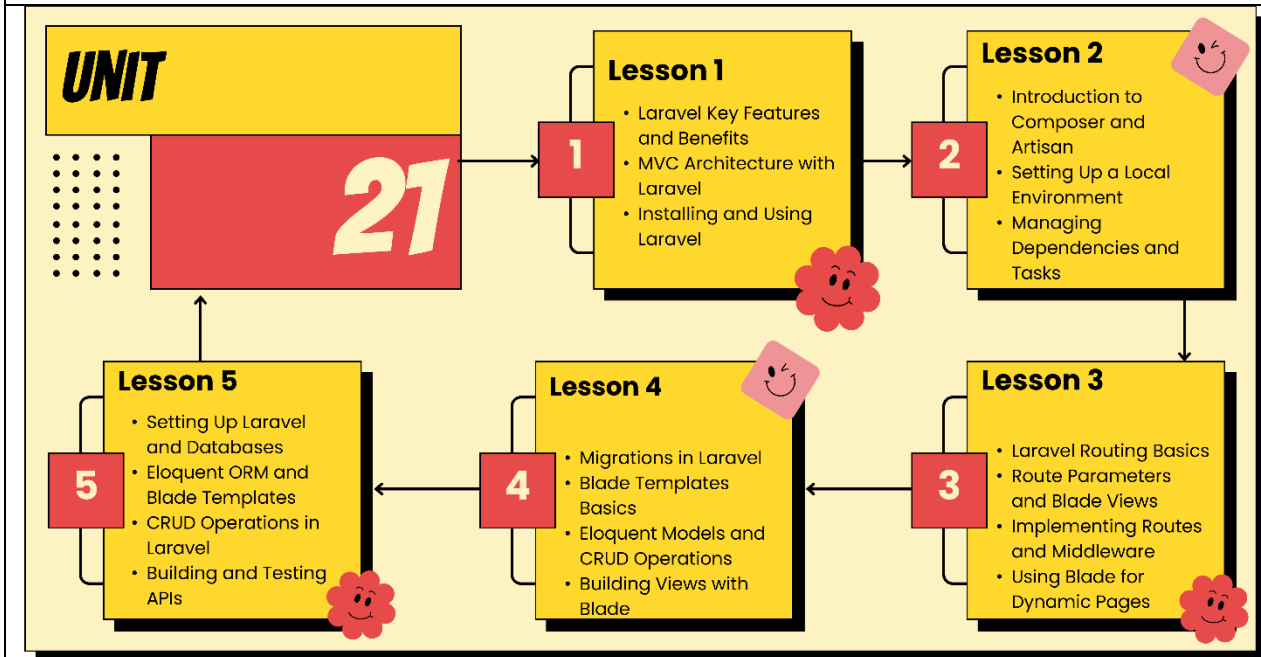
Unit 21

Workshops

Unit	21
Name	Laravel
<u>Goals / Outcomes</u>	➤ <u>Remembering</u> 1. Recall the fundamental concepts and components of the Laravel framework, including its key features, tools, and directory structure. 2. Identify the purposes of core Laravel tools such as Composer, Artisan, Blade templates, and Eloquent ORM. 3. List the basic commands and syntax used for setting up Laravel projects, routes, controllers, and views. ➤ <u>Understanding</u> 1. Explain how Laravel integrates with PHP and its role in web application development. 2. Describe the MVC architecture and how Laravel implements this pattern. 3. Understand the purpose and usage of Laravel's routing system, Blade templating engine, and Eloquent ORM for managing database interactions. 4. Explain how Laravel handles dependencies and simplifies development tasks using Composer and Artisan commands. ➤ <u>Applying</u> 1. Install and configure a Laravel project, including setting up a local development environment and managing dependencies. 2. Write and implement Laravel scripts to manage routes, controllers, and views, and handle HTTP requests. 3. Utilize Laravel's Blade templating engine to create dynamic, reusable views. 4. Develop and manage database schemas using Laravel migrations and seeders. 5. Implement CRUD operations using Eloquent models to interact with databases. ➤ <u>Analyzing</u> 1. Compare Laravel with other PHP frameworks in terms of features, functionality, and performance. 2. Analyze the benefits of using Laravel's routing, templating, and ORM systems for building scalable and maintainable applications. 3. Evaluate different methods for structuring code and organizing application components to enhance reusability and maintainability. 4. Assess the impact of using Laravel's tools (e.g., Composer, Artisan, Blade) on development efficiency and security.

	➤ <u>Evaluating</u> 1. Evaluate the performance and efficiency of Laravel applications under different scenarios. 2. Judge the clarity, maintainability, and scalability of Laravel applications using best practices in coding and architecture. 3. Assess the robustness and security of Laravel applications, particularly in handling HTTP requests, data validation, and database interactions. ➤ <u>Creating</u> 1. Develop complete web applications using Laravel, demonstrating the integration of routes, controllers, models, and views. 2. Build dynamic web pages using Blade templates with layout inheritance, components, and conditional logic. 3. Create and manage database tables using migrations and implement RESTful APIs with Laravel to handle CRUD operations. 4. Design and implement scalable and maintainable web application structures following best practices in Laravel development.	
Knowledge	Code	Description
	TPK22	Analyze and solve common web applications tasks by writing PHP programs
Skill	Code	Description
	TPC5.7	Using controllers and routes for APIs and URLs
	TPC5.8	Creating and using composer packages
	TPC5.9	Create restful services, use an Effect Dependency Array and how to hand errors in data requests

Unit Preface



Workshop 1

Objective: Apply the theoretical concepts from Lesson 1 by creating a Laravel project, understanding its structure, and using Artisan CLI.

Note Close Antivirus

Q1: Setting Up a Laravel Project

Steps:

1. Open Terminal/CMD

Ensure PHP, Composer, and Laravel Installer are installed

In terminal:

```
Composer
```

2. Creating a New Laravel Project:

In terminal:

```
composer global require laravel/installer  
laravel new laravel-first-app
```

Wait for dependencies to install.

Choose the appropriate options during project installation.

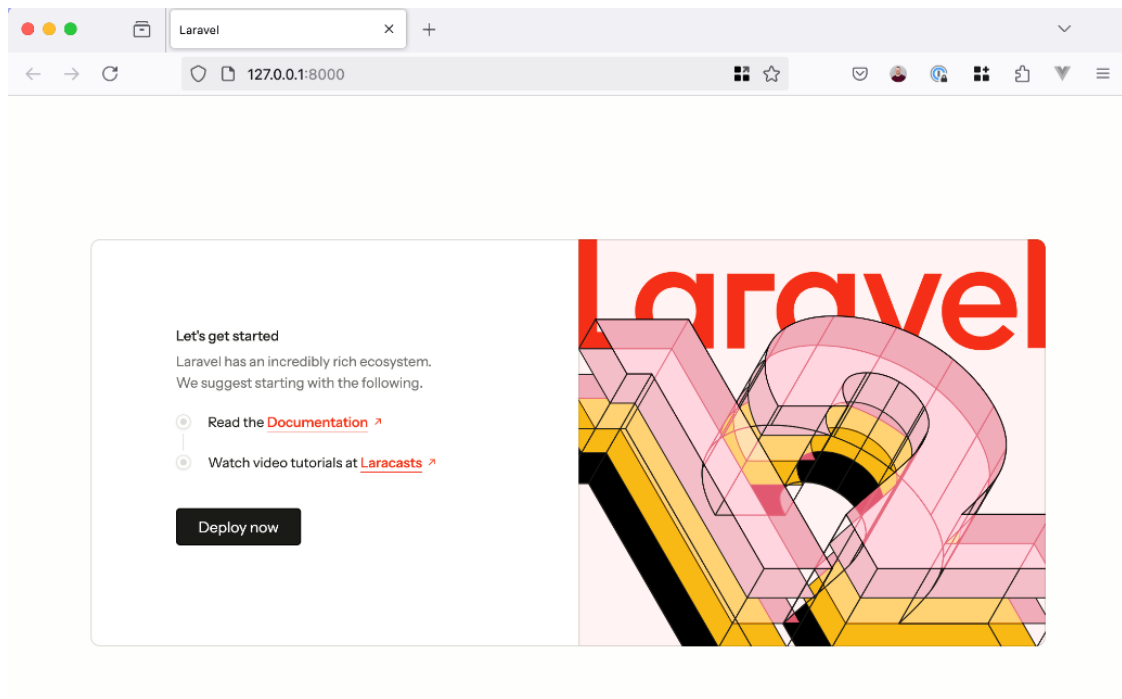
```
Which starter kit would you like to install? [None]  
Which testing framework do you prefer? [Pest]: Pest  
Which database will your application use? [SQLite]: [mysql] MySQL  
Default database updated. Would you like to run the default database  
migrations? (yes/no) [yes]:> yes  
Would you like to run npm install and npm run build? (yes/no): > yes
```

3. Navigate to the Project & Start the Server:

In terminal:

```
cd laravel-first-app  
php artisan serve or composer run dev
```

Open your browser at: <http://localhost:8000>.



Q2: Understanding Laravel's Folder Structure

Task: Explore key directories and modify a file.

1. **Open the project in a code editor (VS Code).**
2. **Modify the Welcome Page:**
 - Go to: `resources/views/welcome.blade.php`.
 - Change any text (e.g., "Welcome to my first Laravel app!").
 - Save the file and refresh the browser to see changes.

Q 3: Using Artisan CLI

Task: Create a Controller and Route.

1. Generate a Controller:

In terminal:

```
php artisan make:controller HelloController
```

File created at: app/Http/Controllers/HelloController.php.

2. Add a Method in the Controller:

```
public function hello(){  
    return "Hello from Laravel's first practical lesson!";  
}
```

3. Create a Route:

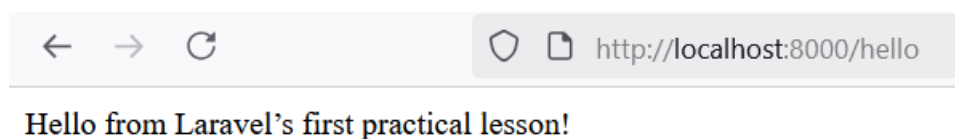
Open: routes/web.php. and Add:

```
use App\Http\Controllers\HelloController;  
Route::get('/hello', [HelloController::class, 'hello']);
```

4. Test the Route:

Visit: <http://localhost:8000/hello>

Expected output: Hello from Laravel's first practical lesson!.



Q 4: Implementing MVC

Task: Create a Model, View, and Controller interaction.

1. Generate a Post Model + Migration:

In terminal:

```
php artisan make:model Post -m
```

-m creates a migration file.

2. Define the Database Table:

Open: database/migrations/..._create_posts_table.php.

Update the up() method:

```
Schema::create('posts', function (Blueprint $table) {  
    $table->id();  
    $table->string('title');  
    $table->text('content');  
    $table->timestamps();  
});
```

3. Run Migrations:

In terminal:

```
php artisan migrate
```

4. Update the Controller:

In HelloController.php:

```
use App\Models\Post;
```

```
public function posts(){  
    $posts = Post::all();  
    return view('posts', compact('posts'));  
}
```

5. Add a Route:

In routes/web.php:

```
Route::get('/posts', [HelloController::class, 'posts']);
```

6. Create a View for Posts:

Create: resources/views/posts.blade.php. and Add code:

```
<h1>Posts List</h1>  
@foreach($posts as $post)  
    <h3>{{ $post->title }}</h3>  
    <p>{{ $post->content }}</p>  
@endforeach
```

7. Insert Test Data

Add data in database manual

Or Use Tinker (*Optional*)

In `app\Models\Post`;

```
protected $fillable = [ 'title', 'content' ];
```

In terminal:

```
php artisan tinker
```

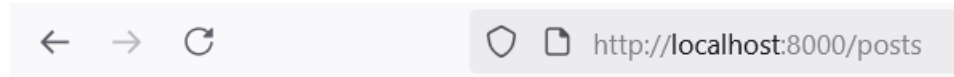
```
use App\Models\Post;
```

```
Post::create(['title' => 'First Post', 'content' => 'This is a test  
post.']);
```

8. Test the Output:

Visit: <http://localhost:8000/posts>

You should see the test post!



Posts List

First Post

This is a test post.

Q 5: Pushing the Project to GitHub

Steps:

1. Create a New Repository on GitHub:

Go to [GitHub](#) → New Repository.

Name: laravel-first-app (or any name).

Do not initialize with a README.

2. Initialize Git Locally:

In terminal:

```
git init
```

3. Commit and Push:

In terminal:

```
git add .
```

```
git commit -m "Initial commit: Laravel project setup"
```

```
git remote add origin https://github.com/username/laravel-first-app.git
```

```
git branch -M main
```

```
git push -u origin main
```

Workshop 2

Objectives By the end of this workshop you will be able to:

- Create and manage Laravel routes
- Build controllers to handle application logic
- Use Blade templates to render views
- Pass data between controllers and views

Exercise 1: Laravel Routing Fundamentals

1.1 What is Routing?

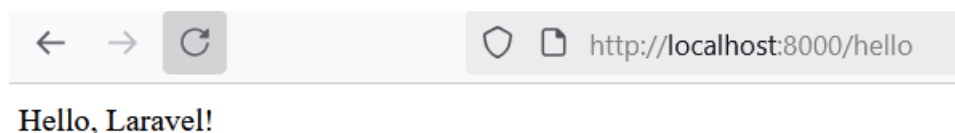
Routing maps URLs to application logic. All web routes are defined in `routes/web.php`.

1.2 Basic Route Example

// routes/web.php

```
Route::get('/hello', function () {  
    return 'Hello, Laravel!';  
});
```

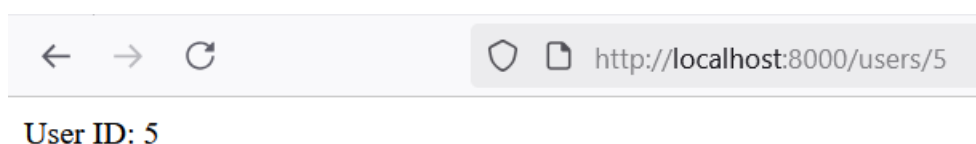
Test it: Visit <http://localhost:8000/hello>



1.3 Route Parameters

```
Route::get('/users/{id}', function ($id) {  
    return "User ID: {$id}";  
});
```

Test it: Visit <http://localhost:8000/users/5>

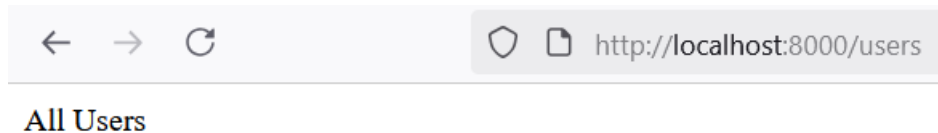


Optional Parameter:

```
Route::get('/users/{id?}', function ($id = null) {  
    return $id ? "User ID: {$id}" : "All Users";  
});
```

Test it: Visit <http://localhost:8000/users/5>

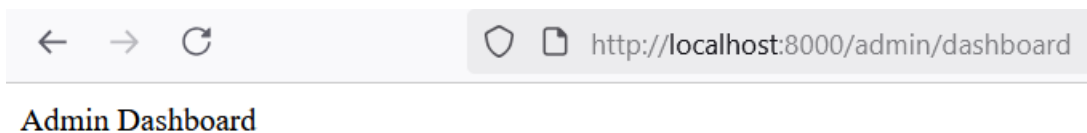
Test it: Visit <http://localhost:8000/users>



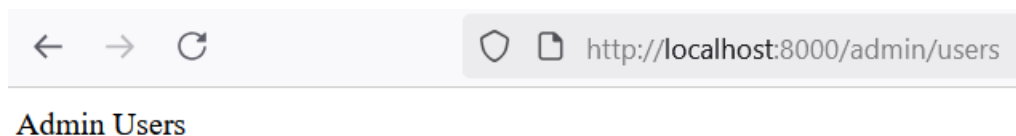
1.4 Route Groups

```
Route::group(['prefix' => 'admin'], function () {  
    Route::get('/dashboard', fn () => "Admin Dashboard");  
    Route::get('/users', fn () => "Admin Users");  
});
```

Access: <http://localhost:8000/admin/dashboard>



Access: <http://localhost:8000/admin/users>



Exercise 2: Controllers

2.1 Creating a Controller

In terminal:

```
php artisan make:controller PostController
```

2.2 Controller Method Example

```
// app/Http/Controllers/PostController.php
```

```
use App\Models\Post;
```

```
public function show($id){  
    return "Post #{ $id }";  
}
```

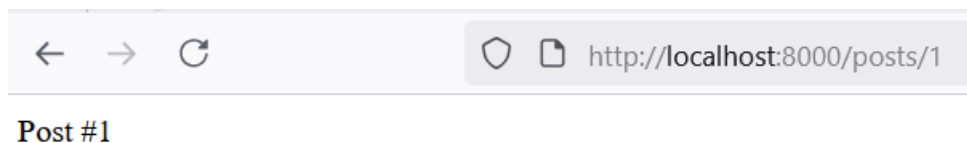
2.3 Linking Routes to Controllers

// routes/web.php

```
use App\Http\Controllers\PostController;
```

```
Route::get('/posts/{id}', [PostController::class, 'show']);
```

Test it: Visit <http://localhost:8000/posts/1>



Exercise 3: Blade Templating

3.1 Creating a Blade View

Create resources/views/posts/show.blade.php:

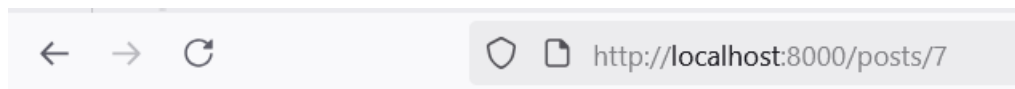
```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Post Details</title>  
</head>  
<body>  
    <h1>Post #{{ $id }}</h1>  
</body>  
</html>
```

3.2 Passing Data to Views

Update the controller:

```
public function show($id){  
    return view('posts.show', ['id' => $id]);  
}
```

Visit <http://localhost:8000/posts/7>



Post #7

Laravel Practice

Mini Project: My Favorite Books App

Project Goal

Create a simple book listing app with:

- A main page showing all books
- Detail pages for each book
- Proper routing and views

Step 1: Set Up the Project

Creating a new Laravel project:

In terminal:

```
laravel new favorite-books  
cd favorite-books  
php artisan serve
```

Visit <http://localhost:8000> to verify setup.

Step 2: Create the Controller

Generate the controller:

In terminal:

```
php artisan make:controller BookController
```

Edit BookController.php (app/Http/Controllers/BookController.php):

```
class BookController extends Controller{
```

```
    // Show all books
```

```
        public function index(){  
            $books = [  
                ['id' => 1, 'title' => 'Harry Potter', 'author' => 'J.K.  
Rowling'],  
                ['id' => 2, 'title' => 'The Hobbit', 'author' => 'J.R.R.  
Tolkien'],  
                ['id' => 3, 'title' => '1984', 'author' => 'George Orwell'],  
            ];  
            return view('books.index', ['books' => $books]);  
        }
```

```
    // Show a single book
```

```
        public function show($id){  
            $books = [  
                1 => ['title' => 'Harry Potter', 'author' => 'J.K. Rowling',  
'summary' => 'A young wizard\'s journey.'],  
                2 => ['title' => 'The Hobbit', 'author' => 'J.R.R. Tolkien',  
'summary' => 'A hobbit\'s adventure.'],  
                3 => ['title' => '1984', 'author' => 'George Orwell',  
'summary' => 'A dystopian future.'],  
            ];  
  
            $book = $books[$id] ?? null;  
            if (!$book) {  
                abort(404); // Show 404 if book doesn't exist  
            }
```



```
}

return view('books.show', ['book' => $book]);

}
```

Step 3: Define Routes

Edit routes/web.php:

```
use App\Http\Controllers\BookController;

Route::get('/books', [BookController::class, 'index']);
Route::get('/books/{id}', [BookController::class, 'show']);
```

Step 4: Create Views

1. Create a books folder In terminal:

```
mkdir resources/views/books
```

2. Create index.blade.php (resources/views/books/index.blade.php):

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
```

```
<meta http-equiv="X-UA-Compatible" content="ie=edge">
```

```
<title>All Books</title>
```

```
<link
```

```
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.7/dist/css/bootstrap.m
in.css" rel="stylesheet" integrity="sha384-
LN+7fdVzj6u52u30Kp6M/trliBMCMKTyK833zpbD+pXdCLuTusPj697
FH4R/5mcr" crossorigin="anonymous">
```

```
</head>
```

```
<body class="container">
```

```

    <h1 class="text-center mt-3">My Favorite Books</h1>
<ul>
    @foreach ($books as $book)
        <li>
            <a href="/books/{{ $book['id'] }}">{{ $book['title'] }} by {{
$book['author'] }}</a>
        </li>
    @endforeach
</ul>
</body>

```

```
</html>
```

3. Create show.blade.php (resources/views/books/show.blade.php):

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">

```

```

    <title>{{ $book['title'] }} Book</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.7/dist/css/bootstrap.mi
n.css" rel="stylesheet" integrity="sha384-
LN+7fdVzj6u52u30Kp6M/trliBMCMKTyK833zpbD+pXdCLuTusPj697
FH4R/5mcr" crossorigin="anonymous">

```

```

</head>
<body class="container">
    <h1 class="text-center mt-3">{{ $book['title'] }}</h1>
    <p><strong>Author:</strong> {{ $book['author'] }}</p>
    <p><strong>Summary:</strong> {{ $book['summary'] }}</p>
    <a href="/books">← Back to list</a>
</body>

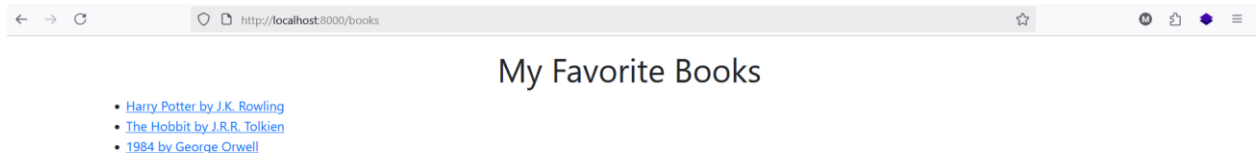
```

</html>

Step 5: Test the App

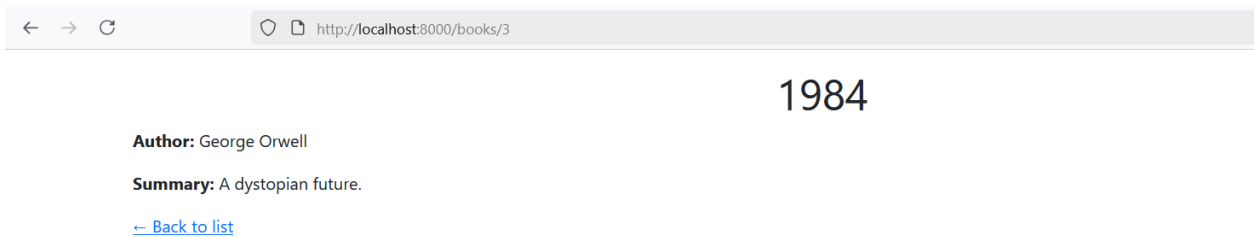
1. Visit the book list:

- Go to <http://localhost:8000/books>
- You should see a list of 3 books.



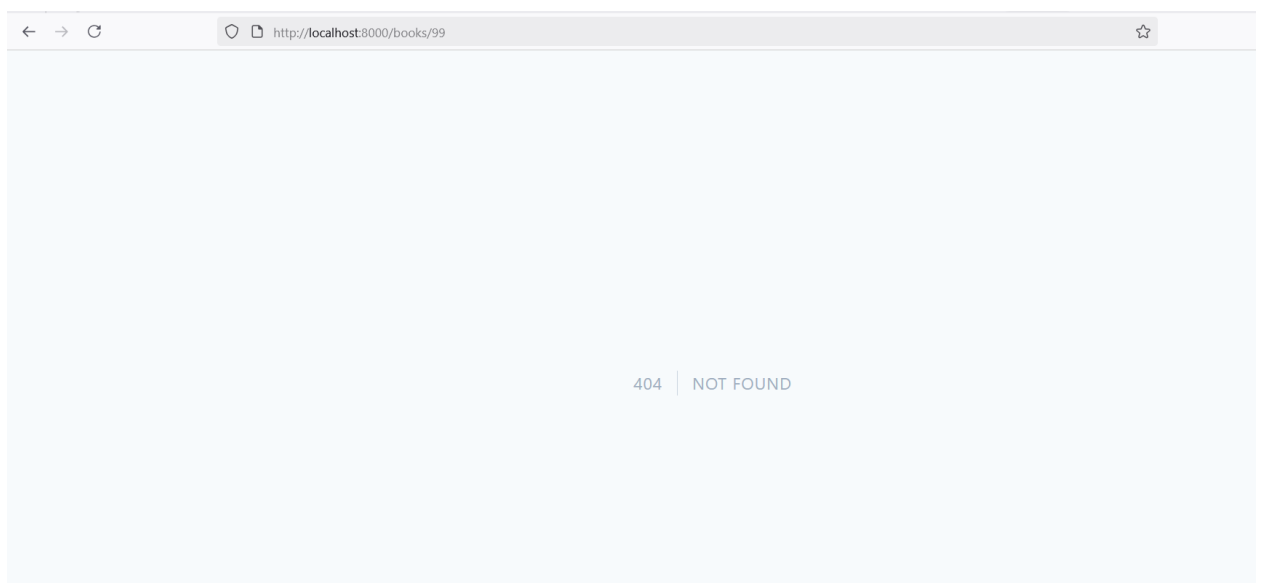
2. View book details:

- Click any book title.
- You'll see details like title, author, and summary.



3. Test error handling:

- Try <http://localhost:8000/books/99> (should show a 404 error).



Workshop 3

Comprehensive Laravel API Development Workshop

Step 1: Create a New Laravel Project

Run the following command to create a new Laravel project:

```
composer create-project laravel/laravel product-api
cd product-api
php artisan serve
```

Step 2: Set Up the Database

1. Create a database product-api in MySQL.
2. Update the .env file with your database credentials:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=product-api
DB_USERNAME=root
DB_PASSWORD=
```

Step 3: Create the Product Model, Migration, Factory, and Seeder

```
php artisan make:model Product -mfs
```

Step 4: Define the Product Migration

Open the generated migration file in the database/migrations folder and define the schema for the products table:

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->text('description')->nullable();
    $table->decimal('price', 8, 2);
    $table->integer('quantity');
    $table->timestamps();
});
```

Run the migration to create the table:

```
php artisan migrate
```

In Product Model Add:

```
protected $fillable = [  
    'name', 'description', 'price', 'quantity'  
];
```

Step 5: Define the Product Factory

Open the ProductFactory file in the database/factories folder and define fake data for testing:

```
return [  
    'name' => $this->faker->word,  
    'description' => $this->faker->sentence,  
    'price' => $this->faker->randomFloat(2, 10, 1000),  
    'quantity' => $this->faker->numberBetween(1, 100),  
];
```

Step 6: Seed the Database

Open the **DatabaseSeeder** file in the database/seeds folder and call the ProductFactory to populate the database:

```
use App\Models\Product;  
public function run(): void{  
    Product::factory()->count(10)->create();  
}
```

Run the seeder:

```
php artisan db:seed
```

Step 7: Create the Product Controller

Run the following command to generate a controller for the Product model:

```
php artisan make:controller ProductController --api
```

This will create a controller with the following methods: index, store, show, update, and destroy.

Create api.php route file

```
php artisan install:api
```

Step 8: Implement the Product Controller

Open the ProductController file in the app/Http/Controllers folder and implement the CRUD operations:

```
use App\Models\Product;
```

```
use Illuminate\Http\Request;
```

```
class ProductController extends Controller{  
    // List all products
```

```
public function index() {  
    return Product::all();  
}
```

```
    // Store a new product
```

```
public function store(Request $request) {  
    return Product::create($request->all());  
}
```

```
    // Show a specific product
```

```
public function show($id){  
    return Product::findOrFail($id);  
}
```

```
    // Update a product
```

```
public function update(Request $request, $id){  
    $product = Product::findOrFail($id);  
    $product->update($request->all()); return $product;  
}
```

```
    // Delete a product
```

```
public function destroy($id) {  
    $product = Product::findOrFail($id);  
    $product->delete();  
    return response()->noContent();  
}
```

```
}
```

Step 9: Define API Routes

Open the api.php file in the routes folder and define the routes for the ProductController:

```
use App\Http\Controllers\ProductController;
```

```
Route::apiResource('products', ProductController::class);
```

In Terminal :

```
php artisan route:list
```

This will automatically create the following routes:

- GET /api/products → index
- POST /api/products → store
- GET /api/products/{id} → show
- PUT/PATCH /api/products/{id} → update
- DELETE /api/products/{id} → destroy

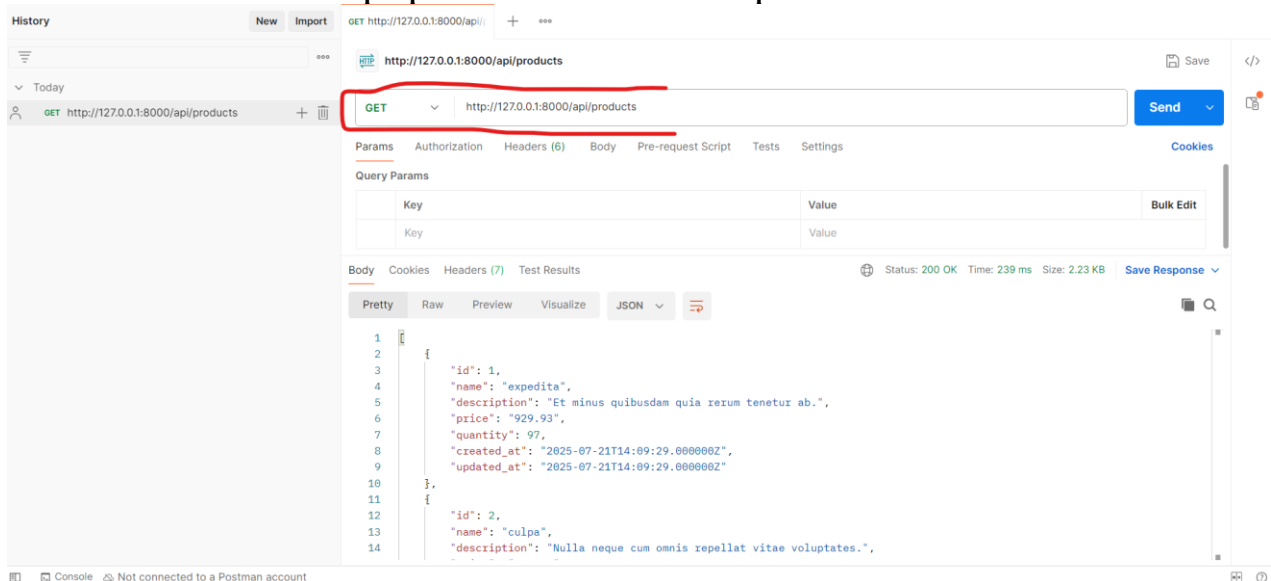
Step 10: Test the API with Postman

Start the Laravel development server:

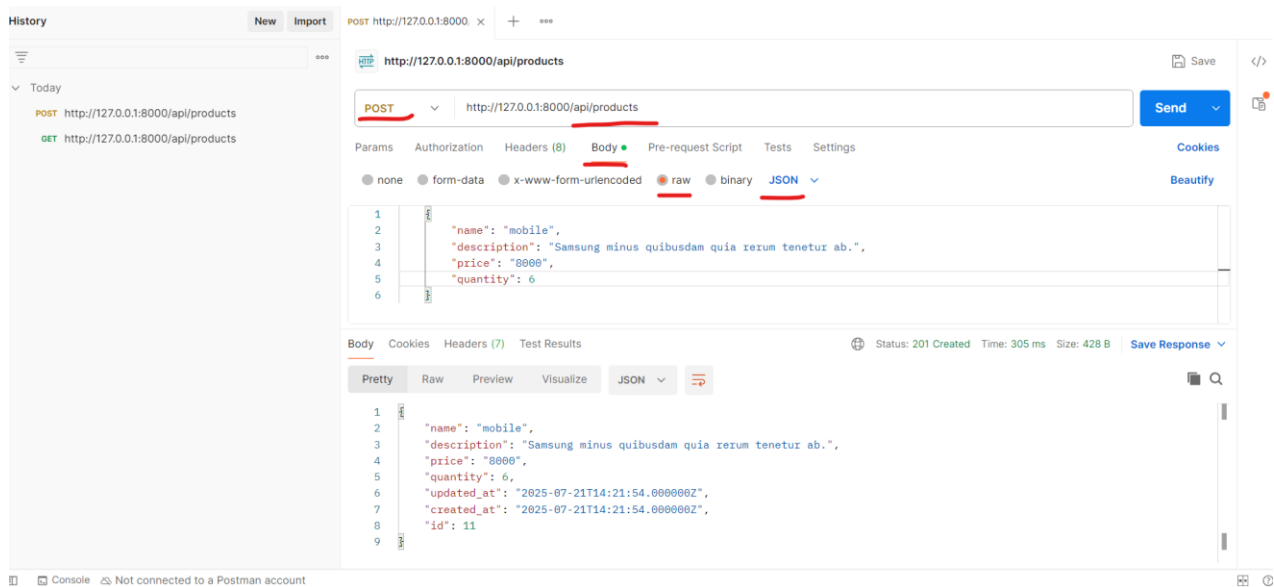
```
php artisan serve
```

1. Use Postman to test all the API endpoints:

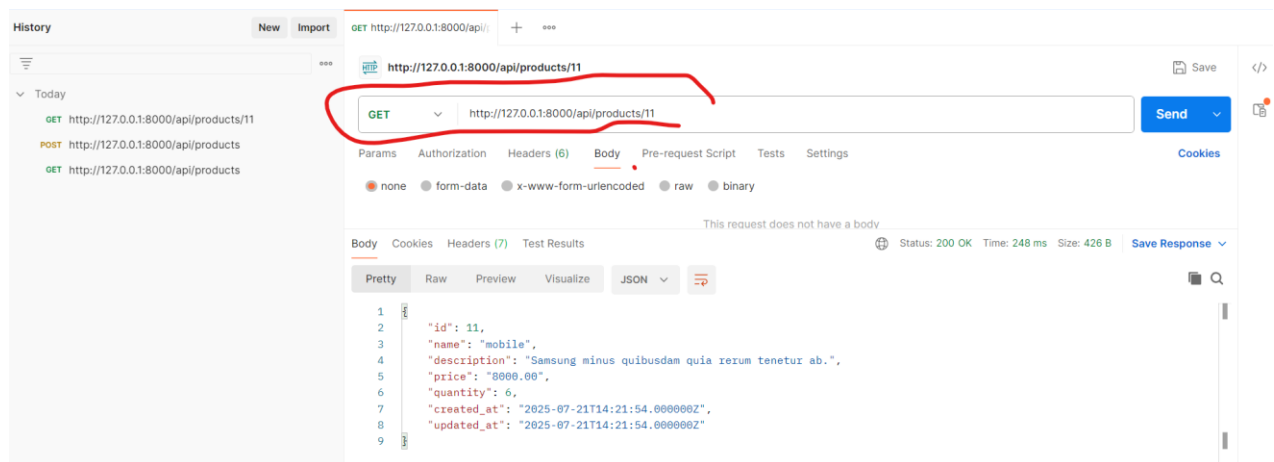
- **GET** /api/products → List all products.



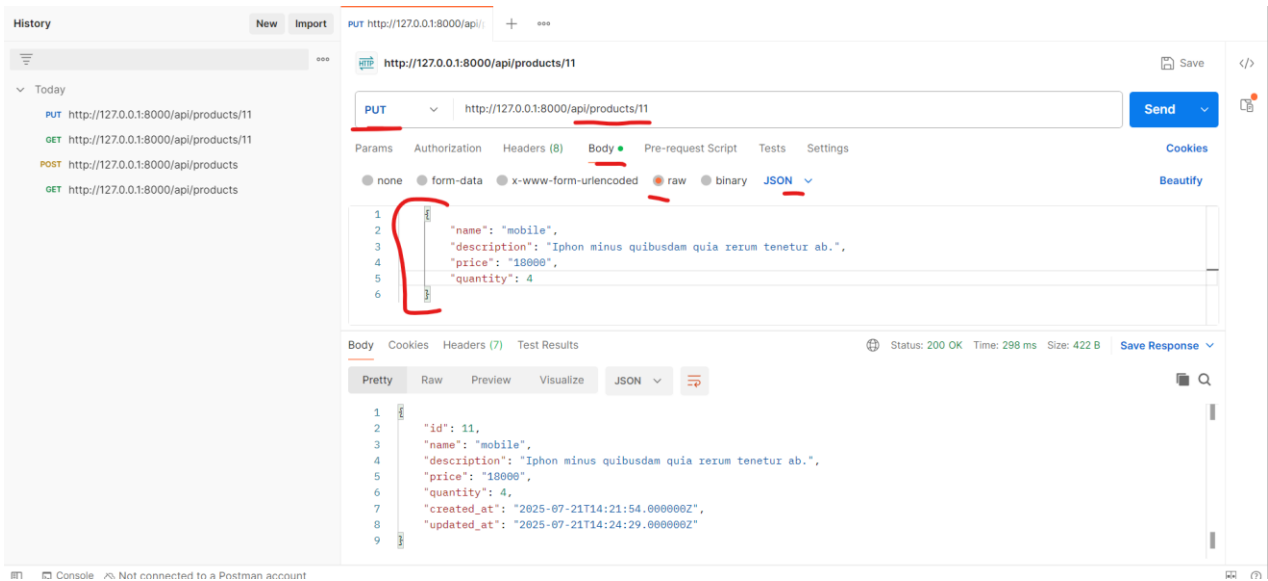
- **POST** /api/products → Add a new product (send JSON data in the request body).



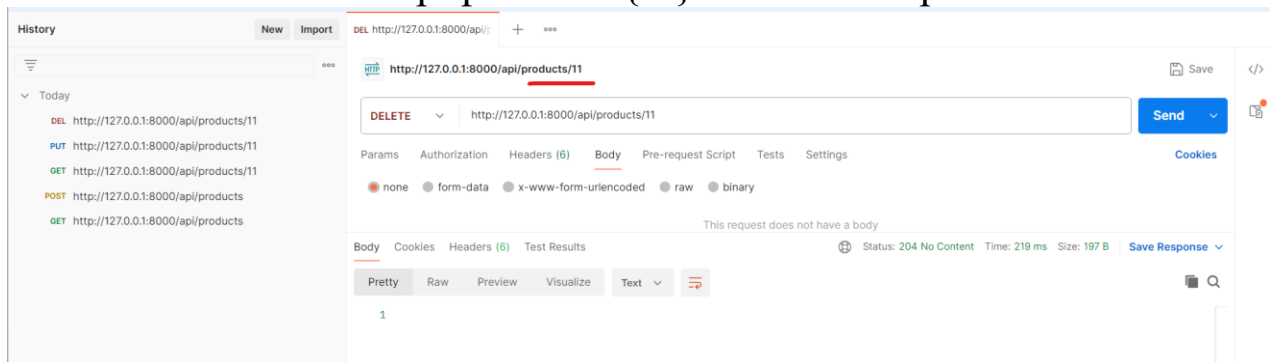
- **GET** `/api/products/{id}` → View details of a specific product.



- **PUT/PATCH** `/api/products/{id}` → Update a product (send JSON data in the request body).



- **DELETE** /api/products/{id} → Delete a product.



Step 11: Verify Functionality

Ensure all CRUD operations work as expected:

- Create new products.
- Retrieve a list of products.
- View details of a specific product.
- Update product details.
- Delete products.

Workshop 4

Complete Laravel CRUD Project with Bootstrap

Step 1: Create a New Laravel Project

Run the following command to create a new Laravel project:

```
composer create-project laravel/laravel laravel-crud  
cd laravel-crud  
php artisan serve
```

Step 2: Set Up the Database

1. Create a database (**laravel-crud**) And Update the .env file with your database credentials:

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=laravel-crud  
DB_USERNAME=root  
DB_PASSWORD=
```

Step 3: Create the Product Model, Migration, Controller

```
php artisan make:model Product -mcr
```

Step 4: Define the Product Migration

Open the generated migration file in the database → migrations folder and define the schema for the products table:

```
Schema::create('products', function (Blueprint $table) {  
    $table->id();  
    $table->string('name');
```

```
$table->text('description')->nullable();  
$table->decimal('price', 8, 2);  
$table->integer('quantity');  
$table->timestamps();  
});
```

Run the migration to create the table:

```
php artisan migrate
```

Step 5: In Product Model Add:

```
protected $fillable = [  
    'name', 'description', 'price', 'quantity'  
];
```

Step 6: Implement the Product Controller

Open the ProductController file in the app/Http/Controllers folder and implement the CRUD operations:

```
use App\Models\Product;
```

```
use Illuminate\Http\Request;
```

```
class ProductController extends Controller{
```

```
    // Display a list of products
```

```
    public function index() {  
        $products = Product::all();  
        return view('products.index', compact('products'));  
    }
```

```
    // Show the form for creating a new product
```

```
public function create(){  
    return view('products.create');  
}
```

// Store a new product

```
public function store(Request $request){  
    $request->validate([  
        'name' => 'required|string|max:255',  
        'description' => 'nullable|string',  
        'price' => 'required|numeric',  
        'quantity' => 'required|integer',  
    ]);  
  
    Product::create($request->all());  
  
    return redirect()->route('products.index')->with('success', 'Product  
created successfully.');
```

// Display details of a specific product

```
public function show($id){  
    $product = Product::findOrFail($id);  
    return view('products.show', compact('product'));  
}
```

// Show the form for editing a product

```
public function edit($id) {
```

```
$product = Product::findOrFail($id);  
return view('products.edit', compact('product'));  
}
```

// Update a product

```
public function update(Request $request, $id) {  
    $product = Product::findOrFail($id);  
  
    $request->validate([  
        'name' => 'required|string|max:255',  
        'description' => 'nullable|string',  
        'price' => 'required|numeric',  
        'quantity' => 'required|integer',  
    ]);  
  
    $product->update($request->all());  
  
    return redirect()->route('products.index')->with('success', 'Product  
updated successfully.');
```

// Delete a product

```
public function destroy($id) {  
    $product = Product::findOrFail($id);  
    $product->delete();  
}
```

```
return redirect()->route('products.index')->with('success', 'Product
deleted successfully.');
```

```
}
```

Step 7: Define Routes

Open the web.php file in the routes folder and define the routes for the ProductController:

```
use App\Http\Controllers\ProductController;

Route::resource('products', ProductController::class);
```

Step 8: Create Blade Views

In resources/views create new folder **products**

In the **products** folder create pages:

index.blade.php , **create.blade.php**, **edit.blade.php**, **show.blade.php**

Create Blade layouts

1. in resources/views **create new folder layouts**
2. in resources/views/layouts/ **create new file app.blade.php:**

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Product Management</title>
```

```
<link
```

```
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
```

```
</head>
```

```
<body>
```

```
  <nav class="navbar navbar-expand-lg navbar-light bg-light">
```

```
    <div class="container-fluid">
```

```
      <a class="navbar-brand" href="{{ route('products.index') }}">Product Management</a>
```

```
    <div class="collapse navbar-collapse">
```

```
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
```

```
        <li class="nav-item">
```

```
          <a class="nav-link" href="{{ route('products.index') }}">Products</a>
```

```
        </li>
```

```
        <li class="nav-item">
```

```
          <a class="nav-link" href="{{ route('products.create') }}">Add Product</a>
```

```
        </li>
```

```
      </ul>
```

```
    </div>
```

```
  </div><!-- container-fluid -->
```

```
</nav>
```

```
<div class="container mt-4">
```

```
  @yield('content')
```

```
</div>
```

```
<script  
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
```

```
</body>
```

```
</html>
```

Step 9 Create Product List Page :

Product Management Products Add Product

Product List

ID	Name	Description	Price	Quantity	Actions
2	culpa	Nulla neque cum omnis repellat vitae voluptates.	\$893.06	88	View Edit Delete
3	sapiente	Officiis et quo molestiae beatae earum optio et.	\$708.22	59	View Edit Delete
4	voluptate	Quo ut blanditiis repellat.	\$218.69	72	View Edit Delete
5	adipisci	Quis pariatur aperiam ipsum et odio.	\$78.79	87	View Edit Delete
6	et	Eum animi accusamus impedit incidunt.	\$243.84	51	View Edit Delete
7	commodi	Ut dolor voluptas dolores perferendis vitae id maiores.	\$575.58	41	View Edit Delete
8	excepturi	Adipisci explicabo consequatur ipsum dolorum et suscipit aut.	\$114.66	78	View Edit Delete
9	tempore	Aut at magni est voluptas atque corporis perspiciatis placeat.	\$881.87	8	View Edit Delete

In resources/views/create **products** folder

Create **resources/views/products/index.blade.php**:

```
@extends('layouts.app')
```

```
@section('content')
```

```
<h1>Product List</h1>
```

```
<table class="table table-striped">
```

```
<thead>
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Name</th>
```

```
<th>Description</th>
```

```
<th>Price</th>
```

```
<th>Quantity</th>
```

```
<th>Actions</th>
```

```
</tr>
```



مدارس وي
للتكنولوجيا
التطبيقية


```

</thead>
<tbody>
  @foreach ($products as $product)
    <tr>
      <td>{{ $product->id }}</td>
      <td>{{ $product->name }}</td>
      <td>{{ $product->description }}</td>
      <td>{{ $product->price }}</td>
      <td>{{ $product->quantity }}</td>
      <td>
        <a href="{{ route('products.show', $product->id) }}" class="btn btn-info btn-sm">View</a>
        <a href="{{ route('products.edit', $product->id) }}" class="btn btn-warning btn-sm">Edit</a>
        <form action="{{ route('products.destroy', $product->id) }}"
method="POST" style="display:inline;">
          @csrf
          @method('DELETE')
          <button type="submit" class="btn btn-danger btn-sm">Delete</button>
        </form>
      </td>
    </tr>
  @endforeach
</tbody>
</table>
@endsection

```

Step 10: Add Product Page:

Product Management Products Add Product

Add Product

Name

Description

Price

Quantity

Submit

Create resources/views/products/create.blade.php:

```
@extends('layouts.app')
@section('content')
<h1>Add Product</h1>
<form action="{{ route('products.store') }}" method="POST">
    @csrf
    <div class="mb-3">
        <label for="name" class="form-label">Name</label>
        <input type="text" class="form-control" id="name" name="name"
required>
    </div>
    <div class="mb-3">
        <label for="description" class="form-label">Description</label>
        <textarea class="form-control" id="description"
name="description"></textarea>
    </div>
```

```

<div class="mb-3">
    <label for="price" class="form-label">Price</label>
    <input type="number" step="0.01" class="form-control" id="price"
name="price" required>
</div>
<div class="mb-3">
    <label for="quantity" class="form-label">Quantity</label>
    <input type="number" class="form-control" id="quantity"
name="quantity" required>
</div>
<button type="submit" class="btn btn-primary">Submit</button>
</form>
@endsection

```

Step 11: Edit Product Page:

Create resources/views/products/edit.blade.php:

Product Management Products Add Product

Edit Product

Name

Description

Price

Quantity

Update

```
@extends('layouts.app')
```

```
@section('content')
```

```
<h1>Edit Product</h1>
```

```

<form action="{{ route('products.update', $product->id) }}"
method="POST">
    @csrf
    @method('PUT')
    <div class="mb-3">
        <label for="name" class="form-label">Name</label>
        <input type="text" class="form-control" id="name" name="name"
value="{{ $product->name }}" required>
    </div>
    <div class="mb-3">
        <label for="description" class="form-label">Description</label>
        <textarea class="form-control" id="description"
name="description">{{ $product->description }}</textarea>
    </div>
    <div class="mb-3">
        <label for="price" class="form-label">Price</label>
        <input type="number" step="0.01" class="form-control" id="price"
name="price" value="{{ $product->price }}" required>
    </div>
    <div class="mb-3">
        <label for="quantity" class="form-label">Quantity</label>
        <input type="number" class="form-control" id="quantity"
name="quantity" value="{{ $product->quantity }}" required>
    </div>
    <button type="submit" class="btn btn-primary">Update</button>
</form>
@endsection

```

Step 12: Product Details Page:

Create resources/views/products/show.blade.php:

Product Management Products Add Product

Product Details

sapiente

Officiis et quo molestiae beatae earum optio et.

Price: \$708.22

Quantity: 59

Edit

Delete

```
@extends('layouts.app')

@section('content')
<h1>Product Details</h1>
<div class="card">
    <div class="card-body">
        <h5 class="card-title">{{ $product->name }}</h5>
        <p class="card-text">{{ $product->description }}</p>
        <p class="card-text"><strong>Price:</strong> ${{ $product->price }}</p>
        <p class="card-text"><strong>Quantity:</strong> {{ $product->quantity }}</p>
        <a href="{{ route('products.edit', $product->id) }}" class="btn btn-warning">Edit</a>
        <form action="{{ route('products.destroy', $product->id) }}"
method="POST" style="display:inline;">
            @csrf
            @method('DELETE')
            <button type="submit" class="btn btn-danger">Delete</button>
        </form>
```

</div>

</div>

@endsection

Test all CRUD operations:

- Add new products.
- Edit existing products.
- View product details.
- Delete products.

WORKSHOP



مدارس وي
للتكنولوجيا
التطبيقية

Workshop 5

Laravel Restaurant Management System

Table of Contents

Section 1: Installing Laravel and Project Setup

- Installing Laravel Installer
- Creating a New Laravel Project
- Configuring the Environment
- Installing and Testing Laravel Breeze Authentication

Section 2: Setting Up Authentication with Breeze

- Installing Breeze
- Testing Register/Login
- Creating Home Page for Public View

Section 3: Database Design and Migrations

- Designing Category and Meal Tables
- Creating Migrations with Models & Controllers
- Running Migrations and Viewing Tables

Section 4: Building Admin Dashboard Layout

- Creating a Layout File with Bootstrap
- Adding Navigation, Dashboard, and Blade Structure

Section 5: Category Management

- Implementing CategoryController (CRUD)
- Building Blade Views for Category Management
- Slug Generation & Validation

Section 6: Meal Management

- MealController with Image Upload
- Create/Edit/Delete Meals
- Meal-Category Relationship
- Validations and File Handling

Section 7: Display Meals on Home Page

- Showing Meals with Categories
- Displaying Images
- Using Bootstrap Cards

Section 1: Installing Laravel and Project Setup

By the end of this section, students will be able to:

- Install Laravel globally
- Creating a new Laravel project
- Configure environment variables
- Set up database connection
- Start the local development server

Tip: Close Antivirus and Run Xampp

1.1 Install Laravel

Open your terminal then run:

```
composer global require laravel/installer
```

1.2 Create a New Laravel Project

Go to the location where you want to install the project.

Open your terminal then run:

```
laravel new restaurant
```

During the installation, choose:

- **Starter kit:** None
- **Testing framework:** Pest
- **Database:** MySQL
- **Run default migrations:** No
- **Run npm install and build:** Yes

1.3 Navigate and Start Laravel Server

Open your terminal then run:

```
cd restaurant  
composer run dev
```

Open in browser:

```
http://localhost:8000
```

1.4 Configure .env File

Open .env in your project folder and update:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=restaurant
DB_USERNAME=root
DB_PASSWORD=
```

Tip: Make sure your MySQL service is running, and the restaurant database exists.

Section 2: Installing Authentication with Laravel Breeze

By the end of this section, students will be able to:

- Understand Laravel Breeze and why it's useful
- Install and configure Breeze with Blade views
- Test user registration, login, and logout
- Prepare for protected admin routes

2.1 What is Laravel Breeze?

Laravel Breeze is a minimal, simple starter kit that provides:

- User registration
- Login/Logout
- Blade templates and Tailwind (we'll use **Blade only**)

It's perfect for anyone who wants **quick authentication** without a heavy frontend framework.

2.2 Install Breeze

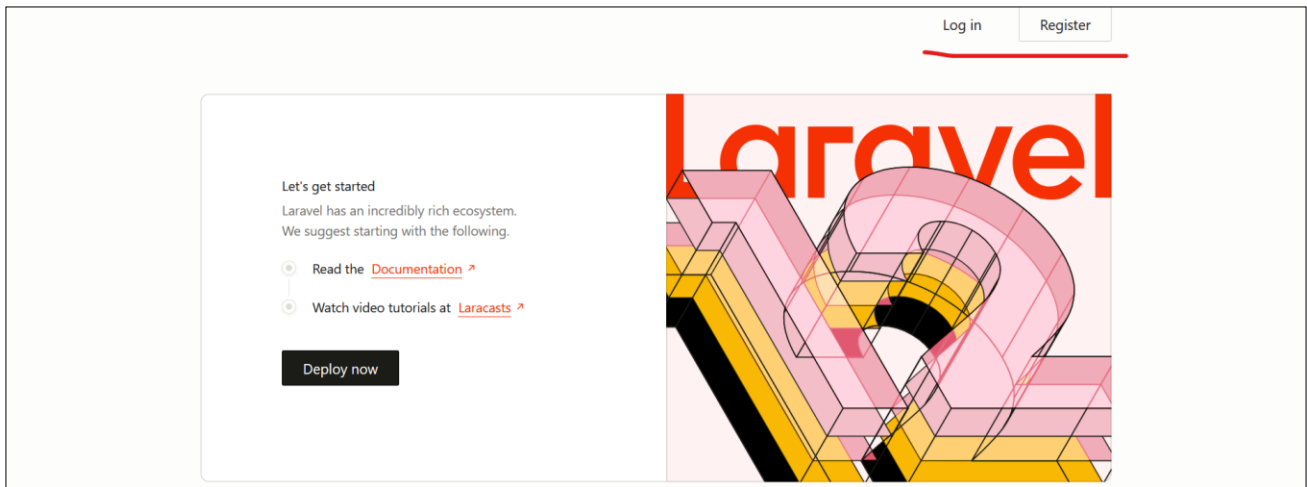
Run the following in the terminal (from your project root):

Open your terminal then run:

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install
php artisan migrate
```

composer run dev (stop and run again)

Tip: If npm install fails due to **antivirus/firewall**, disable it temporarily.



2.3 Test Authentication Pages

Now open your browser and test:

Register page: <http://localhost:8000/register>

Login page: <http://localhost:8000/login>

Use this test user to register:

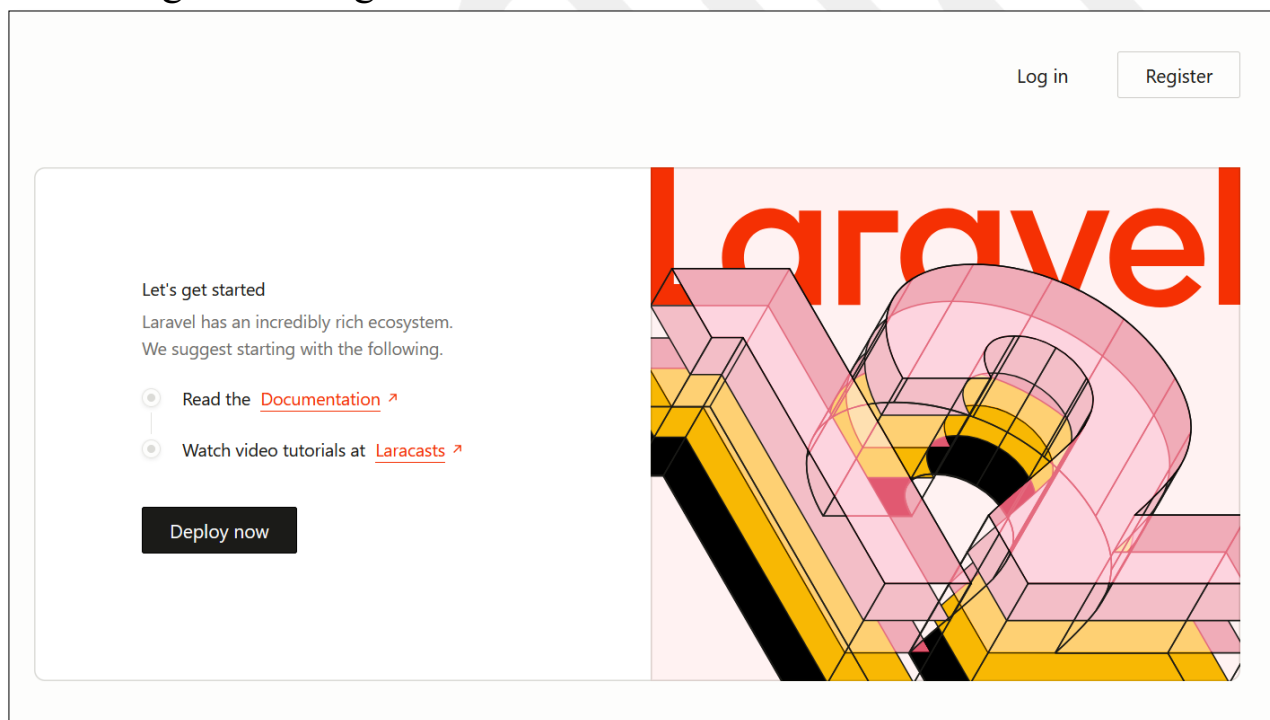
Field	Value
Name	admin
Email	admin@laravel.com
Password	12345678

You can now login and logout as the admin.

Test to register user :admin admin@laravel.com 12345678



Test to logout and login



2.4 Enable Protected Routes

Later in the project, we will group routes for managing categories and meals under this:

```
Route::middleware('auth')->group(function () {  
    // Admin dashboard, categories, meals  
});
```

This ensures only logged-in users can access the admin system.

2.5 Create the Home Page Controller

To set up a public home page:

In terminal:

```
php artisan make:controller HomeController
```

In HomeController.php:

```
public function index() {  
    return view('home');  
}
```

Create view:

In resources/views/, add home.blade.php with simple content:

```
<h1>Home Page</h1>
```

In routes/web.php:

```
use App\Http\Controllers\HomeController;  
Route::get('/', [HomeController::class, 'index'])->name('home');
```

Section 3: Database Design & Migrations

By the end of this section, students will be able to:

- Understand how Laravel migrations work
- Create tables for categories and meals
- Define relationships between tables
- Seed example data (optional)

3.1 Planning the Database

For this project, we'll need the following tables:

Categories Table

- id: primary key
- name: category name such as (Pizza, Drinks)
- slug: URL-friendly version of the name
- created_at / updated_at

Meals Table

- id: primary key
- category_id: foreign key
- name: meal name
- description: short text
- price: decimal
- image: file path
- created_at / updated_at

3.2 Create Models with Migrations

In terminal Generate Category model:

```
php artisan make:model Category -mcr
```

In terminal Generate Meal model:

```
php artisan make:model Meal -mcr
```

Tip: The -mcr flag generates:

- m: migration file
- c: controller
- r: resource controller (includes CRUD methods)

3.3 Edit Migration Files

database/migrations/xxxx_create_categories_table.php:

```
Schema::create('categories', function (Blueprint $table) {  
    $table->id();  
    $table->string('name');  
    $table->string('slug')->unique();  
    $table->timestamps();  
});
```

database/migrations/xxxx_create_meals_table.php:

```
Schema::create('meals', function (Blueprint $table) {  
    $table->id();  
    $table->foreignId('category_id')->constrained()-  
>onDelete('cascade');  
    $table->string('name');  
    $table->text('description');  
    $table->double('price');  
    $table->string('image')->nullable();  
    $table->timestamps();  
});
```

3.4 Run Migrations

In the terminal:

```
php artisan migrate
```

You should now have categories and meals tables in your database.

3.5 Define Relationships in Models

In Category.php:

```
class Category extends Model{
```

```
    protected $fillable = ['name', 'slug', 'description'];
```

```
    public function meals(){
        return $this->hasMany(Meal::class);
    }
```

```
} // class Category
```

In Meal.php:

```
class Meal extends Model{
```

```
    protected $fillable = [
        'category_id', 'name', 'slug', 'description', 'price', 'image'
    ];
```

```
    public function category(){
        return $this->belongsTo(Category::class);
    }
```

```
}//class Meal
```

3.6 (Optional) Create Example Categories

You can add test data manually using Tinker:

In the terminal:

```
php artisan tinker
```

```
\App\Models\Category::create([
    'name' => 'Pizza',
    'slug' => 'pizza',
]);
```

Repeat with other categories like Drinks, Burgers, Desserts, etc.

Section 4: Building the Admin Dashboard Layout

By the end of this section, students will be able to:

- Create a shared admin layout using Blade
- Build a Bootstrap navigation sidebar
- Add Logout functionality
- Create a welcome dashboard page

4.1 Create admin-layout.blade.php in layouts folder

In resources/views/layouts/, create a file named: **admin-layout.blade.php**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>@yield('title', 'Admin Dashboard')</title>
  <!-- Bootstrap 5 CSS CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <div class="container">
    <div class="row">
      <!-- Sidebar -->
      <nav class="col-md-3 d-md-block bg-light sidebar py-4">
        <h4 class="text-center">Restaurant Admin</h4>
        <ul class="nav flex-column px-3">
          <li class="nav-item"><a href="{{ route('dashboard') }}" class="nav-
link">Dashboard</a></li>
          <li class="nav-item"><a href="{{ route('categories.index') }}"
class="nav-link">Categories</a></li>
          <li class="nav-item"><a href="{{ route('meals.index') }}"
class="nav-link">Meals</a></li>
          <li class="nav-item">
            <form method="POST" action="{{ route('logout') }}">
              @csrf
              <button class="btn btn-danger mt-3">Logout</button>
            </form>
          </li>
        </ul>
      </nav>
```

```
<!-- Content -->
```

```
<main class="col-md-9 py-4">
    @yield('content')
</main>
```

```
</div> <!-- row -->
```

```
</div> <!-- container -->
```

```
<!-- Bootstrap JS -->
```

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
```

```
</body>
```

```
</html>
```

4.2 Create Admin Dashboard Page

Create View File: In resources/views/dashboard.blade.php

```
@extends('layouts.admin-layout')
```

```
@section('title', 'Dashboard')
```

```
@section('content')
```

```
<h2>Welcome to the Admin Dashboard</h2>
```

```
<p>Select a section from the sidebar to manage categories or meals.</p>
```

```
@endsection
```

Tip: An error will appear in the **routes**, and we will solve the error by adding the following code in **web.php**

4.3 Protect All Admin Pages (in routes/web.php):

```
use App\Http\Controllers\HomeController;
use App\Http\Controllers\CategoryController;
use App\Http\Controllers\MealController;
```

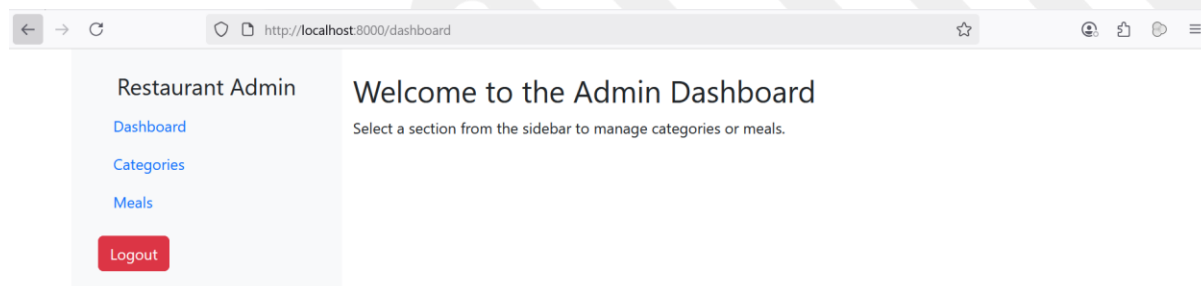
```
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');
```


Make sure all admin routes are grouped inside the auth middleware:

```
Route::middleware('auth')->group(function () {  
    .....  
    // Resource routes  
    Route::resource('categories', CategoryController::class);  
    Route::resource('meals', MealController::class);  
    .....  
});
```

require __DIR__.'./auth.php';

Test to access to dashboard : <http://localhost:8000/dashboard>



After the project is completed, the page will be redesigned with quick links to the main parts of the project and an image that expresses the project's activity, such as adding a category or meal.

Section 5: Category Management (CRUD)

By the end of this section, students will be able to:

- Display a list of categories
- Create a new category
- Edit and delete categories
- Use the admin layout for category pages

5.1 Resource Routes for Categories

Make sure in routes/web.php, inside the auth group:

```
Route::resource('/categories', CategoryController::class);
```

To verify or know the routes, use the following command:

In the terminal:

```
php artisan route:list
```

This auto-generates these routes:

HTTP Verb	URI	Method
GET	/categories	index
GET	/categories/create	create
POST	/categories	store
GET	/categories/{id}	show <i>(not used)</i>
GET	/categories/{id}/edit	edit
PUT/PATCH	/categories/{id}	update
DELETE	/categories/{id}	destroy

5.2 Create Views for Categories (create categories folder)

In resources/views/**categories**/, create these Blade files:

index.blade.php – list all categories

create.blade.php – form to add

edit.blade.php – form to update

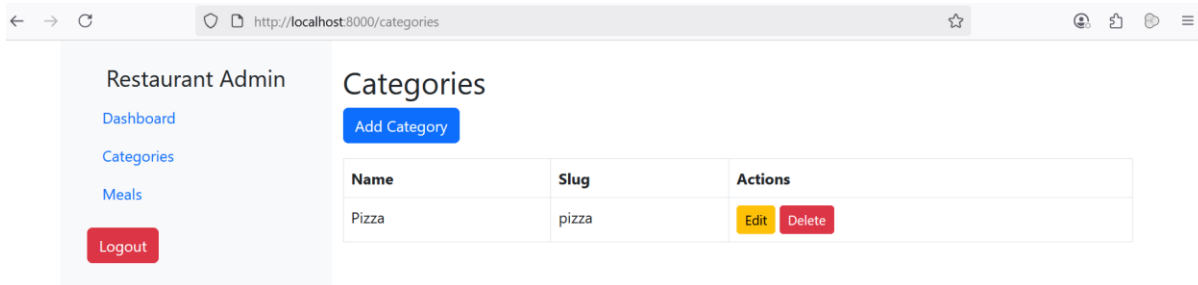
5.3 get and list all categories

In CategoryController Add index method to get categories

```
use App\Models\Category;
```

```
public function index() {  
    $categories = Category::latest()->get();  
    return view('categories.index', compact('categories'));  
}
```

In resources\views\categories\index.blade.php



```
@extends('layouts.admin-layout')
@section('title', 'Categories')
@section('content')
```

```
<h2>Categories</h2>
```

```
<a href="{{ route('categories.create') }}" class="btn btn-primary mb-3"
>Add Category</a>
```

```
<table class="table table-bordered">
```

```
<thead>
```

```
<tr>
```

```
<th>Name</th>
```

```
<th>Slug</th>
```

```
<th>Actions</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
@foreach ($categories as $category)
```

```
<tr>
```

```
<td>{{ $category->name }}</td>
```

```
<td>{{ $category->slug }}</td>
```

```
<td>
```

```
<a href="{{ route('categories.edit', $category) }}" class="btn btn-
warning btn-sm">Edit</a>
```

```
<form action="{{ route('categories.destroy', $category) }}"
method="POST" class="d-inline">
    @csrf @method('DELETE')
```

```
<button class="btn btn-danger btn-sm" onclick="return
confirm('Delete?')">Delete</button>
</form>
```

```
</td>
```

```
</tr>
```

```
@endforeach
```

```
</tbody>
```

```
</table>
```

```
@endsection
```

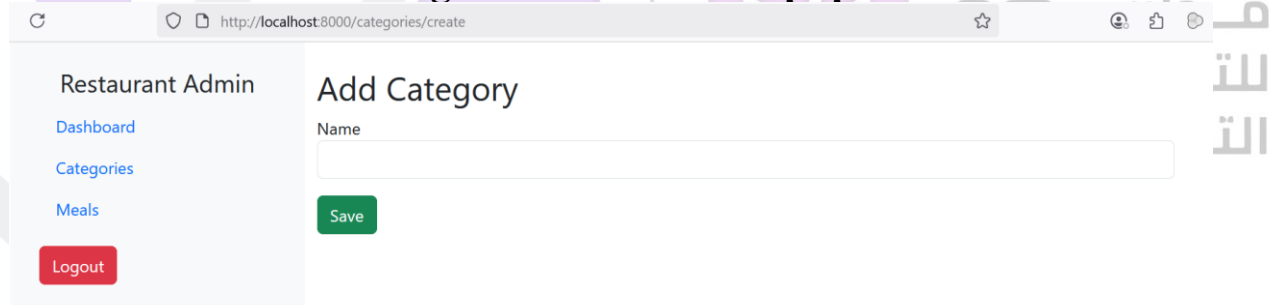
5.4 Add new category CategoryController Methods

In CategoryController Add create methods to display create form

```
use Illuminate\Support\Str;
```

```
public function create() {
    return view('categories.create');
}
```

In resources\views\categories\create.blade.php



```
@extends('layouts.admin-layout')
@section('title', 'Create Category')
@section('content')
```

```
<h2>Add Category</h2>
```

```
<form method="POST" action="{{route('categories.store')}}">
    @csrf
```

```
<div class="mb-3">
```

```

@error('name')
    <div class="alert alert-danger m-3">{{ $message }}</div>
@enderror
<label>Name</label>
<input type="text" name="name"
class="form-control @error('title') is-invalid @enderror" >
</div>

```

```

<button class="btn btn-success" type="submit"> Save </button>
</form>
@endsection

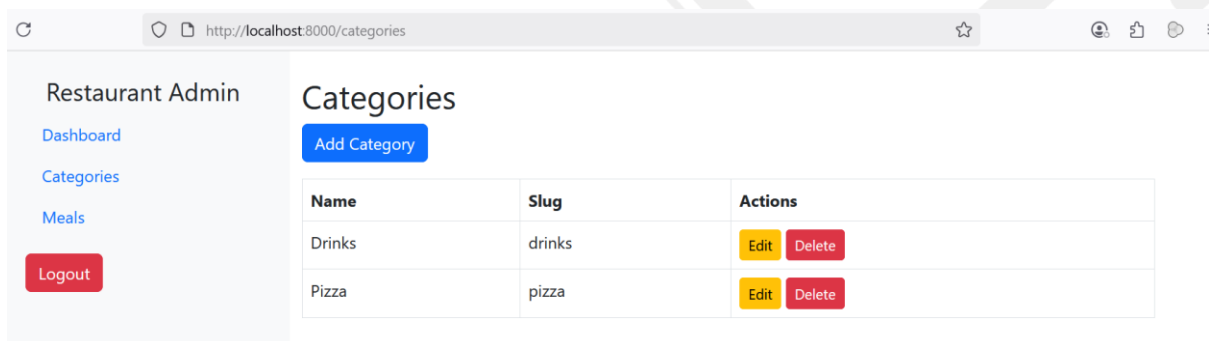
```

In CategoryController Add store method to send data in database

```

public function store(Request $request) {
    $request->validate(['name' => 'required|unique:categories']);
    Category::create([
        'name' => $request->name,
        'slug' => Str::slug($request->name),
    ]);
    session()->flash('success', 'Category created successfully. ');
    return redirect()->route('categories.index');
}

```



In index.blade.php display flash message

Add code above table:

```
@if(session('success'))  
    <div class="alert alert-success">  
        {{ session('success') }}  
    </div>  
@endif
```

```
<table class="table table-bordered">
```

5.5 Update Category

In CategoryController Add edit method to display edit form

```
public function edit(Category $category) {  
    return view('categories.edit', compact('category'));  
}
```

In resources\views\categories\edit.blade.php

```
@extends('layouts.admin-layout')  
@section('title', 'Edit Category')  
@section('content')  
<h2>Edit Category</h2>
```

```
<form method="POST" action="{{route('categories.update',  
$category)}}">
```

```
@csrf
```

```
@method('PUT')
```

```
<div class="mb-3">
```

```
@error('name')
```

```
<div class="alert alert-danger m-3">{{ $message }}</div>
```

```
@enderror
```

```
<label>Name</label>
```

```
<input type="text" name="name"  
value="{{ old('name', $category->name ?? ' ') }}"
```

```

class="form-control @error('title') is-invalid @enderror" >
</div>
<button type="submit" class="btn btn-success">Update</button>
</form>
@endsection

```

In CategoryController in Upade method

```

public function update(Request $request, Category $category) {
    $request->validate([
        'name' => 'required|unique:categories,name,' . $category->id
    ]);
    $category->update([
        'name' => $request->name,
        'slug' => Str::slug($request->name),
    ]);
    session()->flash('success', 'Category Updated successfully.');
```

```

    return redirect()->route('categories.index');
}

```

In CategoryController in destroy method

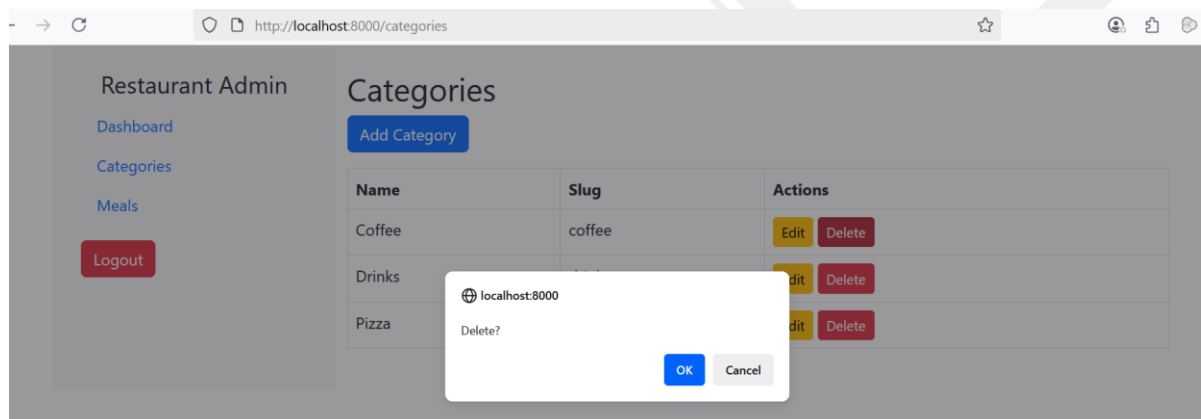
```

public function destroy(Category $category) {
    $category->delete();
    session()->flash('success', 'Category Deleted successfully.');
```

```

    return redirect()->route('categories.index');
}

```



Section 6: Meal Management (CRUD with Image Upload)

By the end of this section, students will be able to:

- Build a full CRUD for meals
- Upload and display meal images
- Connect meals to categories

6.1 Resource Routes for Meals

Make sure in routes/web.php, inside the auth group:

```
Route::resource('/meals', MealController::class);
```

To verify or know the routes, use the following command:

In the terminal:

```
php artisan route:list
```

6.2 Views for Meals (create meals folder)

Folder: resources/views/meals/

Create files:

```
index.blade.php  
create.blade.php  
edit.blade.php
```

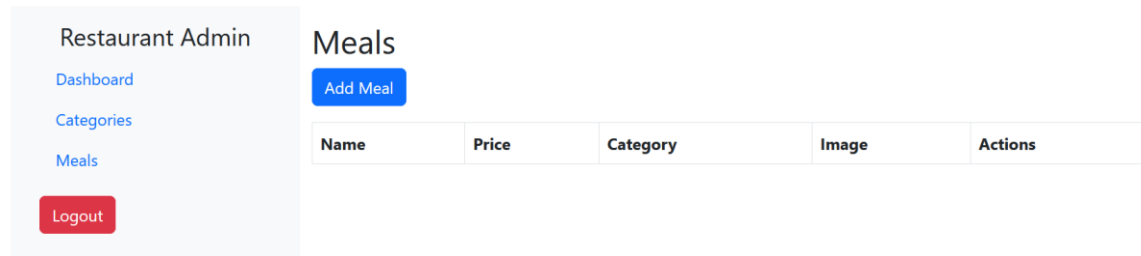
6.3 get and list all meals

In **MealController** Add index method to get meals

```
use App\Models\Meal;  
use App\Models\Category;
```

```
public function index() {  
    $meals = Meal::with('category')->latest()->get();  
    return view('meals.index', compact('meals'));  
}
```


In resources\views\meals\index.blade.php



```
@extends('layouts.admin-layout')
@section('title', 'Meals')
@section('content')
<h2>Meals</h2>
```

```
<a href="{{ route('meals.create') }}" class="btn btn-primary mb-3"> Add
Meal</a>
```

```
@if(session('success'))
    <div class="alert alert-success">
        {{ session('success') }}
    </div>
@endif
```

```
<table class="table table-bordered">
    <thead>
        <tr>
            <th>Name</th><th>Price</th><th>Category</th><th>Image</th><th>
Actions</th>
        </tr>
    </thead>
    <tbody>
        @foreach ($meals as $meal)
            <tr>
```

```
                <td>{{ $meal->name }}</td>
                <td>{{ $meal->price }}</td>
                <td>{{ $meal->category->name }}</td>
```

```
            <td>
```

```

    @if($meal->image)
      
    @endif
  </td>

```

```

<td>
  <a href="{{ route('meals.edit', $meal) }}" class="btn btn-warning btn-sm">Edit</a>

  <form action="{{ route('meals.destroy', $meal) }}" method="POST"
class="d-inline">
    @csrf @method('DELETE')

    <button class="btn btn-danger btn-sm" onclick="return
confirm('Delete?')">Delete</button>
  </form>
</td>

```

```

</tr>
@endforeach
</tbody>
</table>
@endsection

```



مدارس وي
للتكنولوجيا
التطبيقية

6.4 Add new category MealController Methods

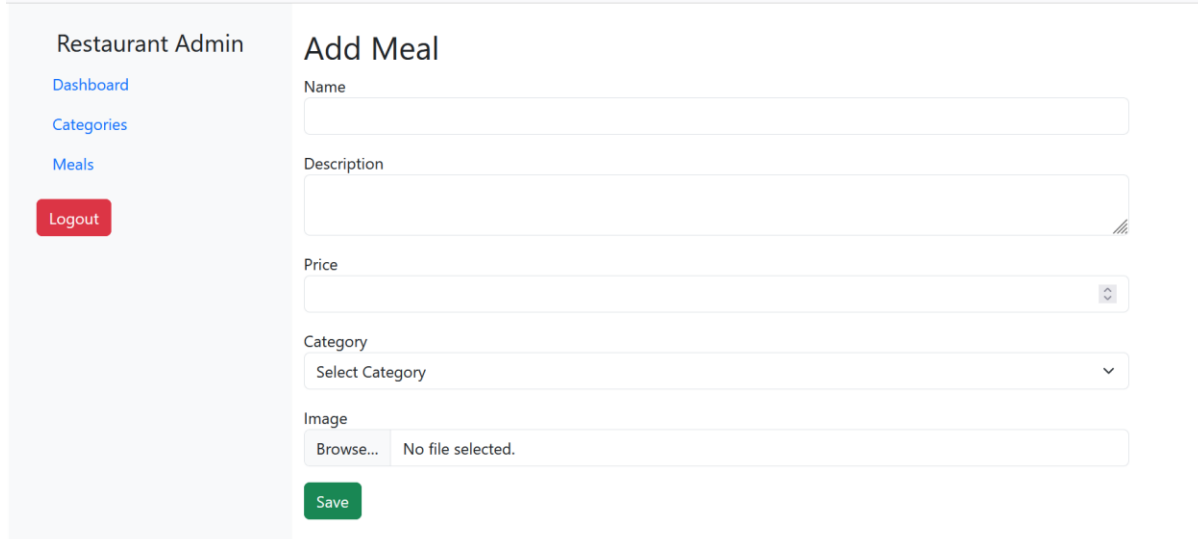
In MealController Add create methods to display create form

```

public function create() {
    $categories = Category::all();
    return view('meals.create', compact('categories'));
}

```

In resources\views\meals\create.blade.php



Restaurant Admin

- Dashboard
- Categories
- Meals
- Logout

Add Meal

Name

Description

Price

Category

Select Category

Image

Browse... No file selected.

Save

```
@extends('layouts.admin-layout')
```

```
@section('title', 'Create Meal')
```

```
@section('content')
```

```
<h2>Add Meal</h2>
```

```
@if ($errors->any())
```

```
<div class="alert alert-danger">
```

```
<ul>
```

```
@foreach ($errors->all() as $error)
```

```
<li>{{ $error }}</li>
```

```
@endforeach
```

```
</ul>
```

```
</div>
```

```
@endif
```

```
<form method="POST" action="{{ route('meals.store') }}"  
enctype="multipart/form-data">
```

```
@csrf
```

```
<div class="mb-3">
```

```
<label>Name</label>
```

```
<input type="text" name="name" class="form-control" >
```

```
</div>
```

```
<div class="mb-3">
  <label>Description</label>
  <textarea name="description" class="form-control"></textarea>
</div>
```

```
<div class="mb-3">
  <label>Price</label>
  <input type="number" name="price" class="form-control" >
</div>
```

```
<div class="mb-3">
  <label>Category</label>
  <select name="category_id" class="form-select" >
    <option>Select Category</option>
    @foreach($categories as $category)
      <option value="{{ $category->id }}">
        {{ $category->name }}
      </option>
    @endforeach
  </select>
</div>
```

```
<div class="mb-3">
  <label>Image</label>
  <input type="file" name="image" class="form-control">
</div>
```

```
<button type="submit" class="btn btn-success">Save</button>
```

```
</form>
```

```
@endsection
```

In Terminal

```
php artisan storage:link
```

In MealController Add store method to store data

```
use Illuminate\Support\Facades\Storage;
```

```
public function store(Request $request){
```

```
    $request->validate([
        'name' => 'required',
        'description' => 'required',
        'price' => 'required|numeric',
        'category_id' => 'required|exists:categories,id',
        'image' => 'image|mimes:jpg,png,jpeg|max:2048',
    ]);
```

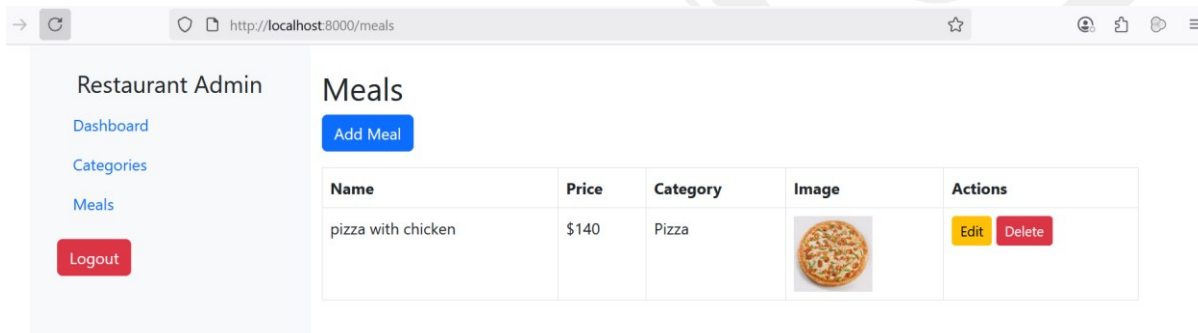
```
    $image = $request->file('image')?->store('meals', 'public');
```

```
    Meal::create([
        'name' => $request->name,
        'description' => $request->description,
        'price' => $request->price,
        'category_id' => $request->category_id,
        'image' => $image,
    ]);
```

```
    session()->flash('success', 'Meal created successfully.');
```

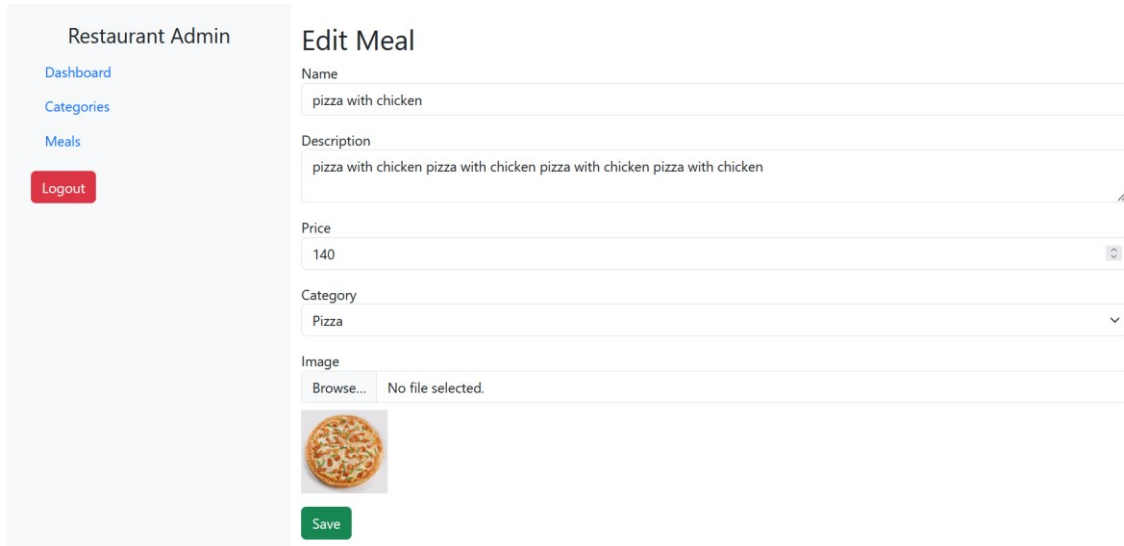
```
    return redirect()->route('meals.index');
```

```
}
```



In MealController In edit method

```
public function edit(Meal $meal) {  
    $categories = Category::all();  
    return view('meals.edit', compact('meal', 'categories'));  
}
```



Restaurant Admin

Dashboard

Categories

Meals

Logout

Edit Meal

Name
pizza with chicken

Description
pizza with chicken pizza with chicken pizza with chicken pizza with chicken

Price
140

Category
Pizza

Image
Browse... No file selected.



Save

In resources\views\meals\edit.blade.php

```
@extends('layouts.admin-layout')  
@section('title', 'Edit Meal')  
@section('content')
```

```
<h2>Edit Meal</h2>
```

```
@if ($errors->any())  
    <div class="alert alert-danger">  
        <ul>  
            @foreach ($errors->all() as $error)  
                <li>{{ $error }}</li>  
            @endforeach  
        </ul>  
    </div>  
@endif
```

```
<form method="POST" action="{{route('meals.update', $meal->id)}}"  
}} " enctype="multipart/form-data">
```

```
@csrf
```

```
@method('PUT')
```

```
<div class="mb-3">  
  <label>Name</label>  
  <input type="text" name="name" value="{{ $meal->name }}"  
class="form-control" >  
</div>
```

```
<div class="mb-3">  
  <label>Description</label>  
  <textarea name="description" class="form-control">{{ $meal-  
>description }}</textarea>  
</div>
```

```
<div class="mb-3">  
  <label>Price</label>  
  <input type="number" name="price" value="{{ $meal->price }}"  
class="form-control" >  
</div>
```

```
<div class="mb-3">  
  <label>Category</label>  
  <select name="category_id" class="form-select" >  
    @foreach ($categories as $category)  
      <option value="{{ $category->id }}" {{ $category->id == $meal-  
>category_id ? 'selected' : " }}">{{ $category->name }}</option>  
    @endforeach  
  </select>  
</div>
```

```
<div class="mb-3">  
  <label>Image</label>  
  <input type="file" name="image" class="form-control">  
  @if(isset($meal) && $meal->image)
```



```

    @endif
</div>
```

```
<button type="submit" class="btn btn-success">Update</button>
</form>
@endsection
```

In CategoryController in update method

```
public function update(Request $request, Meal $meal){
```

```
    $request->validate([
        'name' => 'required',
        'description' => 'required',
        'price' => 'required|numeric',
        'category_id' => 'required|exists:categories,id',
        'image' => 'image|mimes:jpg,png,jpeg|max:2048',
    ]);
```

```
    if ($request->hasFile('image')) {
        Storage::disk('public')->delete($meal->image);
        $image = $request->file('image')->store('meals', 'public');
        $meal->image = $image;
    }
```

```
    $meal->update([
        'name' => $request->name,
        'description' => $request->description,
        'price' => $request->price,
        'category_id' => $request->category_id,
    ]);
```

```
    session()->flash('success', 'Meal created successfully. ');
    return redirect()->route('meals.index');
} // update
```


In CategoryController in destroy method

```
public function destroy(Meal $meal) {  
    Storage::disk('public')->delete($meal->image);  
    $meal->delete();  
    session()->flash('success', 'Meal deleted successfully.');
```

```
    return redirect()->route('meals.index');
```

```
} // destroy
```

Section 7: Customer Frontend – View Menu & Filter by Category

By the end of this section, students will be able to:

- Displays meals with images and prices
- Filters meals by selected category

Add Methods to HomeController

```
use App\Models\Meal;  
use App\Models\Category;
```

```
class HomeController extends Controller{  
    // Show all meals on the homepage
```

```
    public function index(){  
        $meals = Meal::with('category')->latest()->get();  
        $categories = Category::all();  
        return view('home', compact('meals', 'categories'));
```

```
    }  
  
    // Filter meals by category
```

```
    public function category($id){  
        $meals = Meal::where('category_id', $id)->get();  
        $categories = Category::all();  
        return view('home', compact('meals', 'categories'));
```

```
    }
```

in routes/web.php

Make sure these routes go **outside the auth middleware group**, since they are public:

```
use App\Http\Controllers\HomeController;
```

```
Route::get('/', [HomeController::class, 'index'])->name('home');  
Route::get('/category/{id}', [HomeController::class, 'category'])->  
>name('category.filter');
```

In resources\views\home.blade.php

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Restaurant</title>  
  <link  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c  
ss" rel="stylesheet">  
</head>  
<body>  
  <div class="container">  
    <h2 class="text-center my-4">Our Menu</h2>
```

```
<div class="text-center mb-4">  
  @foreach($categories as $category)  
    <a href="{{ route('category.filter', $category->id) }}" class="btn btn-  
outline-dark m-1">  
      {{ $category->name }}  
    </a>  
  @endforeach  
  <a href="{{ route('home') }}" class="btn btn-outline-secondary m-  
1">Show All</a>  
</div>
```

```
<div class="row">  
  @forelse($meals as $meal)  
    <div class="col-md-4 mb-4">
```

```

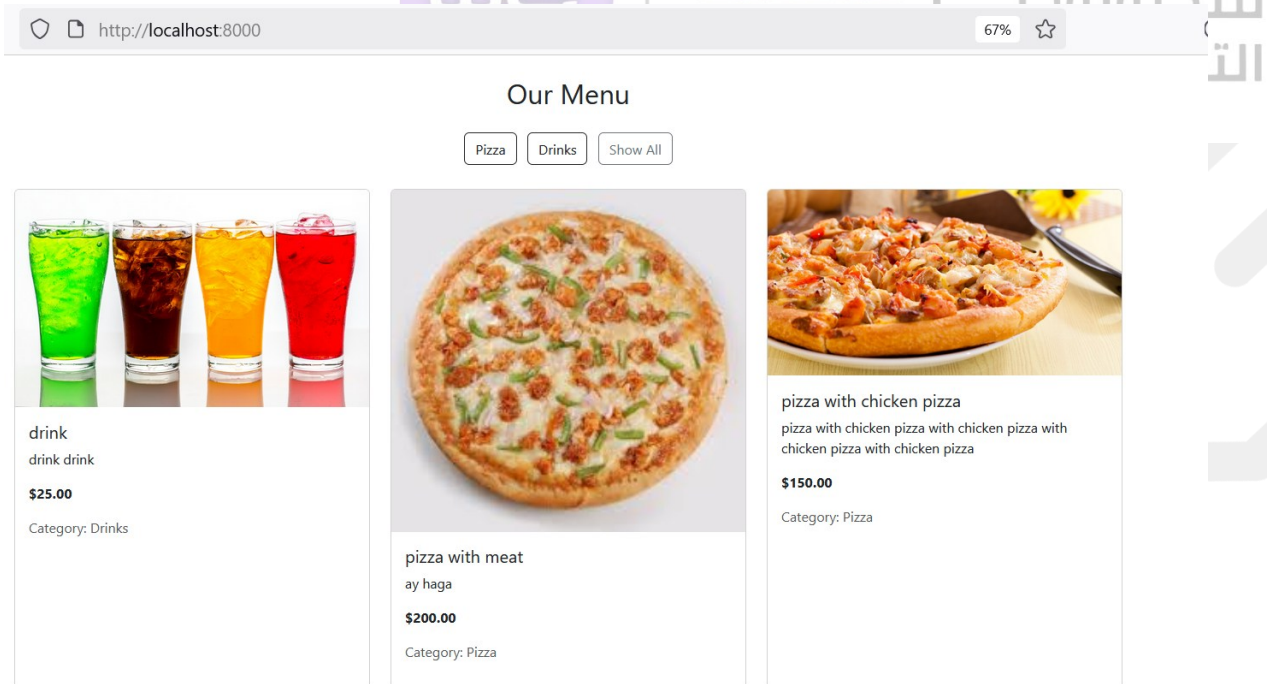
<div class="card h-100">
  @if($meal->image)
    name }}">
  @endif
  <div class="card-body">
    <h5 class="card-title">{{ $meal->name }}</h5>
    <p class="card-text">{{ Str::limit($meal->description, 100) }}</p>
    <p class="fw-bold">${{ number_format($meal->price, 2) }}</p>
    <p class="text-muted">Category: {{ $meal->category->name }}</p>
  </div>
</div><!-- card -->

```

```

</div> <!-- col -->
@empty
  <p class="text-center">No meals available for this category.</p>
@endforelse
</div><!-- row -->
</div> <!-- container -->
</body>
</html>

```



Some challenges for students

- Create show page for meal with logic code
- Change design

